

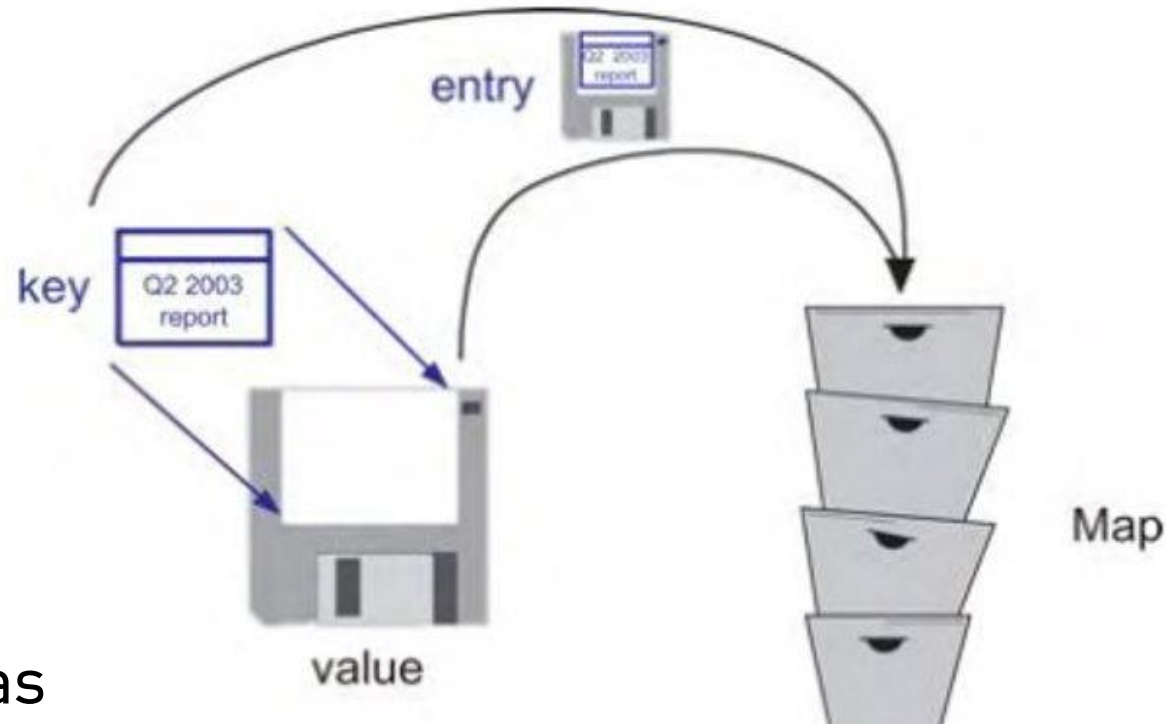
Algoritmos y Estructuras de Datos

1er semestre - 2026

Mapas

- Un “mapa” nos permite almacenar elementos de manera tal que puedan ser rápidamente ubicados utilizando claves.
- Cada elemento contiene valiosa información adicional a su clave.
- Un mapa almacena pares *clave-valor* (k,v) , tales que cada clave es única.

Mapas



- Claves: etiquetas
- Valores (discos)
- Entrada en el mapa(caja): disco etiquetado
- Las claves pueden ser usadas luego para buscar y recuperar los discos

TDA Mapa

- Almacena una colección de objetos en la forma **clave-valor**
 - *tamaño()*
 - *estaVacio()*
 - *recuperar(k)*
 - *poner(k,v)*
 - *eliminar(k)*
 - *claves()*
 - *valores()*
 - *elementos()*

Diccionarios

- Como los mapas, también almacenan pares **clave-valor**
- El diccionario permite múltiples entradas con la misma clave
- Dos tipos:
 - Ordenados
 - desordenados

TDA Diccionario

- *tamaño()*
- *estaVacio()*
- *buscar(k)*
- *buscarTodos(k)*
- *insertar(k,v)*
- *eliminar(e)*
- *elementos()*
 - *Cada elemento tiene **getKey** y **getValue***

Contrato entre hashCode() y equals()

- Cada vez que se invoca en el mismo objeto más de una vez durante la ejecución de una aplicación Java, **hashCode()** debe devolver el mismo entero, siempre que no se modifique la información utilizada en las comparaciones de igualdad en el objeto.
- Este entero no necesita permanecer consistente entre dos ejecuciones de la misma aplicación o programa.
- Si dos objetos son iguales de acuerdo con el método **equals()**, entonces llamar a **hashCode()** en cada uno de los dos objetos debe producir el mismo resultado entero.
- No se requiere que si dos objetos son desiguales de acuerdo con **equals()**, entonces llamar a **hashCode()** en cada uno de los dos objetos debe producir resultados enteros distintos.
- [Java hashCode\(\) and equals\(\) Methods - HowToDoInJava](#)

hashCode()

- Indica en un documento cuáles son las implementaciones estándar de hashCode() para los siguientes tipos:
 - Int
 - String
 - Float
 - Object
- ¿cómo definirías el hashCode() para números de cédula, sabiendo que los que vas a considerar son uruguayos de entre 20 y 25 años de edad?

Implementación para integer

- En Java, `int` es un tipo primitivo, por lo que no tiene métodos propios. El tipo envoltorio correspondiente es `Integer`.
- La implementación estándar de `Integer.hashCode()` devuelve directamente el valor entero almacenado.

```
Integer numero = 25;  
System.out.println(numero.hashCode()); // 25
```

Implementación para String

- La clase `String` calcula su `hashCode()` usando una fórmula polinómica basada en los caracteres de la cadena.
- Para una cadena `s` de longitud `n`, la fórmula es:
- $s[0] * 31^{(n-1)} + s[1] * 31^{(n-2)} + \dots + s[n-1]$

```
int hash = 0;

for (int i = 0; i < texto.length(); i++) {
    hash = 31 * hash + texto.charAt(i);
}
```

Implementación para floats

- La clase `Float` calcula su `hashCode()` a partir de la representación binaria del número en formato IEEE 754.
- Conceptualmente, la implementación es equivalente a:

```
public int hashCode() {  
    return Float.floatToIntBits(value);  
}
```

hashCode() vs equals()

- ¿cuál es la relación entre hashCode y equals?
- ¿cómo funcionan estos métodos cuando se inserta o busca en un hashMap?
- ¿Cuáles son los parámetros posibles para el constructor del HashMap?

hashCode() vs equals()

- En Java, equals() define si dos objetos son considerados **iguales lógicamente**.
- hashCode() devuelve un número entero que Java usa para ubicar rápidamente un objeto dentro de estructuras hash como:
- HashMap
- HashSet
- Hashtable

```
if (a.equals(b)) {  
    a.hashCode() == b.hashCode(); // debe ser true  
}
```

Implementación para Object

- Para definir el `hashCode()` de un objeto, se deben usar los mismos atributos que definen su identidad lógica en `equals()`.
- No necesariamente es “el atributo más representativo”, sino el atributo o conjunto de atributos que hacen que ese objeto sea considerado único o igual a otro.

Implementación para Object – Caso 1

- Un solo atributo identifica al objeto
- Por ejemplo, si una Persona se identifica por su cédula:

```
@Override
public int hashCode() {
    return cedula.hashCode();
}

@Override
public boolean equals(Object obj) {
    if (!(obj instanceof Persona)) {
        return false;
    }

    Persona otra = (Persona) obj;
    return this.cedula.equals(otra.cedula);
}
```

Implementación para Object – Caso 1

- varios atributos identifican al objeto
- Por ejemplo, si un EstudianteCurso se identifica por cédula + código de curso:

```
@Override
public int hashCode() {
    return Objects.hash(cedula, codigoCurso);
}

@Override
public boolean equals(Object obj) {
    if (!(obj instanceof EstudianteCurso)) {
        return false;
    }

    EstudianteCurso otro = (EstudianteCurso) obj;
    return Objects.equals(this.cedula, otro.cedula)
        && Objects.equals(this.codigoCurso, otro.codigoCurso);
}
```


Implementación para Object

- Para un objeto, `hashCode()` se define a partir de los atributos que determinan su igualdad lógica. Si un único atributo identifica al objeto, se puede utilizar el `hashCode()` de ese atributo. Si la identidad depende de varios atributos, se compone un hash combinando los `hashCode()` de todos ellos, por ejemplo mediante `Objects.hash(...)`.
- A tener en cuenta, no conviene utilizar atributos cambiantes como:
 - Email
 - Dirección

API Colecciones JAVA

- Una colección – a veces llamada *contenedor* – es un objeto que agrupa múltiples elementos en una sola unidad.
- Las colecciones se utilizan para almacenar, recuperar, manipular y comunicar los datos agregados.
- El framework de colecciones:
 - Interfaces
 - Implementaciones
 - Algoritmos

¿Cuál de los siguientes no es un beneficio de usar la librería de colecciones?

- a) Reducir el esfuerzo de programación
- b) Incrementar la velocidad y calidad del programa
- c) Reducir el esfuerzo para aprender y usar nuevas APIs (muchas usan las colecciones como entrada y / o salida)
- d) Promover la reutilización del software
- e) Ahorrar en el desarrollo de casos de prueba

API Colecciones – mas comunes

- **Set**

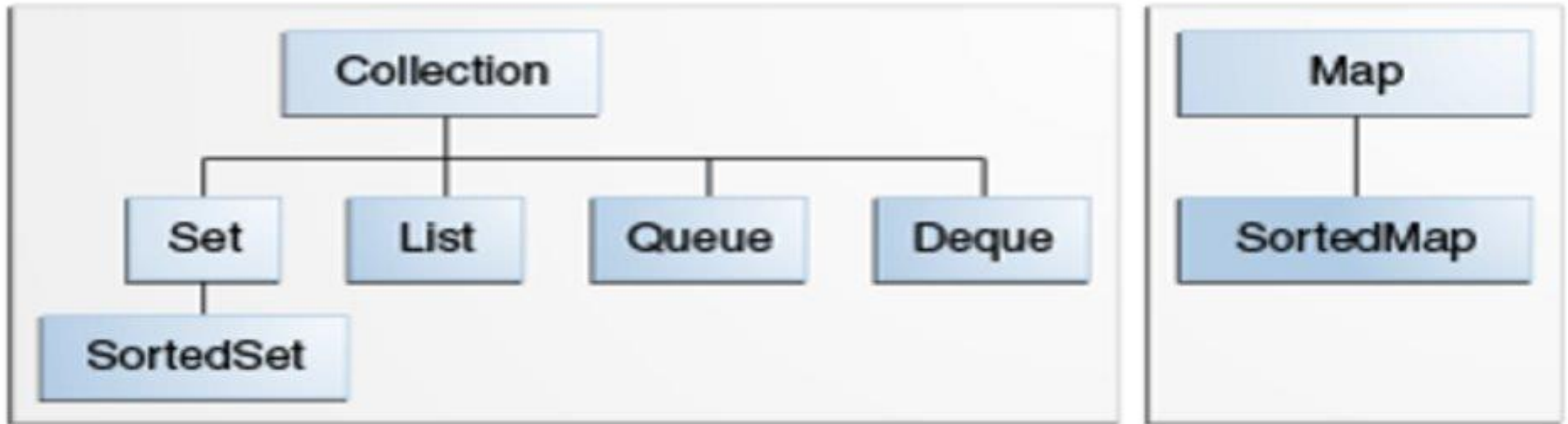
- HashSet
- TreeSet
- LinkedHashSet

- **List**

- ArrayList
- LinkedList

- **Map**

- HashMap
- ConcurrentHashMap
- TreeMap
- LinkedHashMap



The core collection interfaces.

API Colecciones – Interfaz “Collection”

- `int size()`,
- `boolean isEmpty()`,
- `boolean contains(Object element)`,
- `boolean add(E element)`,
- `boolean remove(Object element)`,
- `Iterator<E> iterator()`.
- Recorridos:
 - `for-each`
 - `Iterador`
 - Operaciones agregadas
- “bulk operations”
- Array operations

API Colecciones – Interfaz “Set”

- Colección que no puede tener elementos duplicados
- Modela la abstracción matemática “conjunto”
- Sólo contiene métodos heredados de Collection.
- Implementaciones
 - HashSet
 - TreeSet
 - LinkedHashSet

API Colecciones – Interfaz “List”

- Es una Colección ordenada (secuencia)
- Puede contener elementos duplicados
- Operaciones adicionales (a las de Collection)
 - Acceso por posición (get, set, add, addAll, remove)
 - Búsqueda (indexOf, lastIndexOf)
 - Iteración (extiende la semántica de Iterator – métodos de listIterator)
 - Vista de subrango (método sublist)
- Dos implementaciones:
 - ArrayList
 - LinkedList

API Colecciones – Interfaz “List”

¿qué funcionalidades ofrece el listIterator?

1. hasNext
2. next
3. remove
4. hasPrevious
5. previous

¿cómo se usa? Ejemplo....

API Colecciones – Interfaz “List”

```
List<String> nombres = new ArrayList<>();

nombres.add("Ana");
nombres.add("Luis");
nombres.add("Marta");

ListIterator<String> it = nombres.listIterator();

// Recorrer hacia adelante
while (it.hasNext()) {
    String nombre = it.next();
    System.out.println(nombre);

    if (nombre.equals("Luis")) {
        it.remove();
    }
}

System.out.println(nombres); // [Ana, Marta]

// Recorrer hacia atrás
while (it.hasPrevious()) {
    String nombre = it.previous();
    System.out.println(nombre);
}
```

API Colecciones – Interfaz “List”

- Vista de sub-rangos
- *subList(int fromIndex, int toIndex)*
 - *for (int i = fromIndex; i < toIndex; i++) { ... }*
- Elimina la necesidad de operaciones desubranging explícitas...
 - *Ej: list.subList(fromIndex, toIndex).clear();*

Algoritmos de Listas

- La mayoría de los algoritmos polimórficos de la clase Collections se aplican específicamente a List:
 - sort — ¿cómo funciona? ¿es “estable”?
 - shuffle — ¿qué hace?
 - reverse.
 - rotate — ¿qué hace?
 - swap —.
 - replaceAll
 - fill
 - copy.
 - binarySearch
 - indexOfSubList
 - lastIndexOfSubList

Como implemento una Cola

- Usando la interfaz Queue que me da operaciones basicas como offer, poll, peek

```
Queue<String> cola = new LinkedList<>();

cola.offer("Ana");
cola.offer("Luis");
cola.offer("Marta");

System.out.println("Primero: " + cola.peek());

while (!cola.isEmpty()) {
    System.out.println("Atendiendo: " + cola.poll());
}
```

Como implemento una Cola : Otras opciones

LinkedList

- `Queue<String> cola = new LinkedList<>();`
- Sirve como cola normal.

ArrayDeque

- `Queue<String> cola = new ArrayDeque<>();`
- Suele ser más eficiente que `LinkedList` para colas simples.

PriorityQueue

- `Queue<Integer> cola = new PriorityQueue<>();`
- No respeta FIFO puro. Atiende según prioridad.

Ejemplo PriorityQueue

```
Queue<Tarea> cola = new PriorityQueue<>(  
    (a, b) -> Integer.compare(a.prioridad, b.prioridad)  
);  
  
cola.offer(new Tarea("Responder correo", 3));  
cola.offer(new Tarea("Resolver caída en producción", 1));  
cola.offer(new Tarea("Actualizar documentación", 5));  
  
while (!cola.isEmpty()) {  
    System.out.println(cola.poll());  
}
```

Como implemento una Pila

Uso Stack

- `Stack<String> pila = new Stack<>();`
- Esta interfaz se encuentra en desuso ya que es mas eficiente `ArrayDeque`

```
Deque<String> pila = new ArrayDeque<>();

pila.push("Libro 1");
pila.push("Libro 2");
pila.push("Libro 3");

System.out.println("Pila completa: " + pila);

System.out.println("Elemento en el tope: " + pila.peek());

System.out.println("Sacando: " + pila.pop());
System.out.println("Sacando: " + pila.pop());

System.out.println("Pila final: " + pila);
```


API Colecciones – Interfaz “Map”

- Un Map mapea claves a valores.
- Cada clave puede mapear como máximo a un valor
- Operaciones básicas
 - put
 - get
 - remove
 - containsKey
 - containsValue
 - size
 - empty
- Operaciones “bulk”
 - putAll
 - Clear
- Vistas
 - keySet, entrySet, values

API Colecciones – Interfaz “Map” implementaciones

- HashMap
- ConcurrentHashMap
- TreeMap
- LinkedHashMap

Collection views

- Los métodos de vistas de *Collection* permiten que un *Map* sea visto como una *Collection* de tres formas:
 - `keySet`
 - `values`
 - `EntrySet` – `Entry<k,v>`

Collection views

- Las vistas de *Collection* proveen la *única forma* de iterar sobre un *Map*.

```
for (KeyType key : m.keySet())  
    System.out.println(key);
```

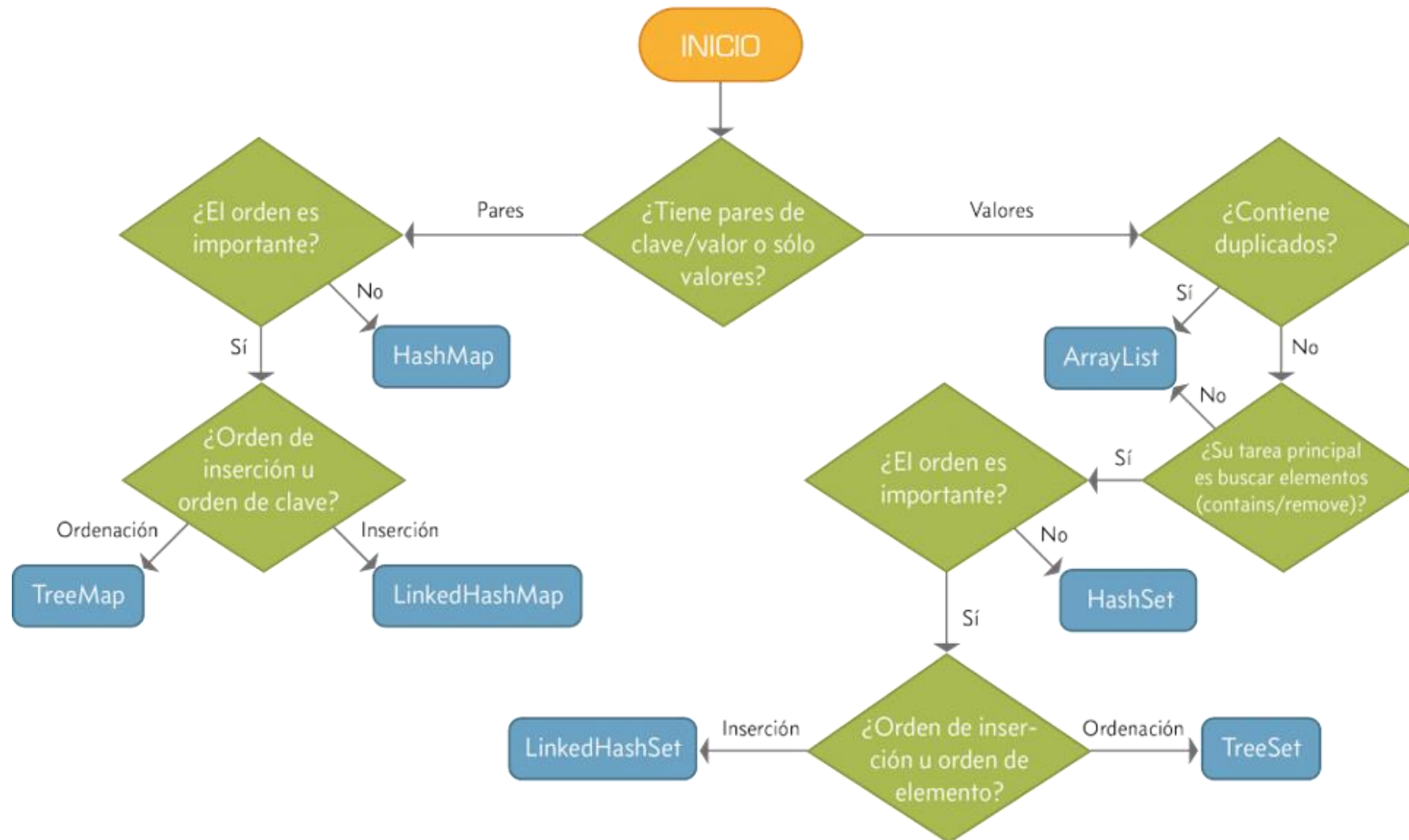
- y con un iterador:

```
for (Iterator<Type> it = m.keySet().iterator(); it.hasNext(); )  
    if (it.next().isBogus()) it.remove();
```

Criterios para elegir la implementación adecuada

- ¿Qué se almacenará en la colección?
 - Objetos (solo valores) o pares clave / valor?
 - Se aceptan datos duplicados?
- Performance ¿Qué acciones se realizarán sobre la colección
¿Cuál es el cometido?
 - Se utilizará para búsqueda o se realizarán constantes inserciones y eliminaciones?
 - Importa el orden de inserción?
 - Es necesario tener los datos ordenados?
- ¿Se trabaja en un ambiente de concurrencia?

Diagrama de decisión para uso de colecciones Java



Interfaces	Implementaciones				
	Hash table	Resizable array	Tree	Linked list	Hash table + Linked list
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue					
Map	HashMap		TreeMap		LinkedHashMap

Property	HashMap	LinkedHashMap	TreeMap
Time Complexity (Big O notation) Get, Put, ContainsKey and Remove method	O(1)	O(1)	O(log n)
Iteration Order	Random	Sorted according to either Insertion Order or Access Order (as specified during construction)	Sorted according to either natural order of keys or comparator (as specified during construction)
Null Keys	allowed	allowed	Not allowed if Key uses Natural Ordering or Comparator does not support comparison on null Keys
Interface	Map	Map	Map, SortedMap and NavigableMap
Synchronization	none, use Collections.synchronizedMap()	None, use Collections.synchronizedMap()	none, use Collections.synchronizedMap()
Data Structure	List of Buckets, if more than 8 entries in bucket then Java 8 will switch to balanced tree from linked list	Doubly Linked List of Buckets	Red-Black Tree (a kind of self-balancing binary search tree) implementation of Binary Tree. This data structure offers O (log n) for Insert, Delete and Search operations and O (n) Space Complexity
Applications	General Purpose, fast retrieval, non-synchronized. ConcurrentHashMap can be used where concurrency is involved.	Can be used for LRU cache, other places where insertion or access order matters	Algorithms where Sorted or Navigable features are required. For example, find among the list of employees whose salary is next to given employee, Range Search, etc.
Requirements for Keys	Equals() and hashCode() needs to be overwritten	Equals() and hashCode() needs to be overwritten	Comparator needs to be supplied for Key implementation, otherwise natural order will be used to sort the keys

Uso de hashMap

- ¿cómo es la estructura interna del hashMap?
(dibújala!!!!)
- ¿cuáles son los parámetros que tiene esta colección?
- ¿cuáles son los valores por defecto?

Collection	Ordering	Random Access	Key-Value	Duplicate Elements	Null Element	Thread Safety
ArrayList	✓	✓	✗	✓	✓	✗
LinkedList	✓	✗	✗	✓	✓	✗
HashSet	✗	✗	✗	✗	✓	✗
TreeSet	✓	✗	✗	✗	✗	✗
HashMap	✗	✓	✓	✗	✓	✗
TreeMap	✓	✓	✓	✗	✗	✗
Vector	✓	✓	✗	✓	✓	✓
Hashtable	✗	✓	✓	✗	✗	✓
Properties	✗	✓	✓	✗	✗	✓
Stack	✓	✗	✗	✓	✓	✓
CopyOnWriteArrayList	✓	✓	✗	✓	✓	✓
ConcurrentHashMap	✗	✓	✓	✗	✗	✓
CopyOnWriteArraySet	✗	✗	✗	✗	✓	✓

Links útiles sobre hashCode enJava

- <https://www.baeldung.com/java-hashcode>
- [https://es.wikipedia.org/wiki/HashCode\(\)_\(Java\)](https://es.wikipedia.org/wiki/HashCode()_(Java))
- [String Hashing - Algorithms for Competitive Programming \(cp-algorithms.com\)](https://cp-algorithms.com/string/string-hashing.html)
- <https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html>